

Chương 5:

QUẢN LÝ BỘ NHỚ

ThS. GVC Tô Oai Hùng

1

1

Nội Dung

1. Sự phân cấp bộ nhớ.
2. Bộ nhớ trong.
3. Vấn đề quản lý bộ nhớ.
4. Mô hình quản lý bộ nhớ thực.
5. Mô hình quản lý bộ nhớ ảo.

ThS. GVC Tô Oai Hùng

2

2

5.1 Sự Phân Cấp Bộ Nhớ

- ❖ Các yêu cầu trong thiết kế bộ nhớ máy tính có thể được tóm tắt trong ba câu hỏi:
 - Dung lượng bao nhiêu?
 - Nhanh thế nào?
 - Giá bao nhiêu?
- ❖ Trong trường hợp lý tưởng, đó là sự cân bằng giữa ba nét đặc trưng ở trên.
- ❖ Nhiều công nghệ được sử dụng để chế tạo bộ nhớ và chúng có các mối quan hệ sau:
 - Thời gian truy xuất nhanh hơn, giá thành của mỗi bit nhớ lớn hơn.
 - Dung lượng lớn hơn, giá thành mỗi bit nhớ nhỏ hơn.

ThS. GVC Tô Oai Hùng

3

3

Sự Phân Cấp Bộ Nhớ

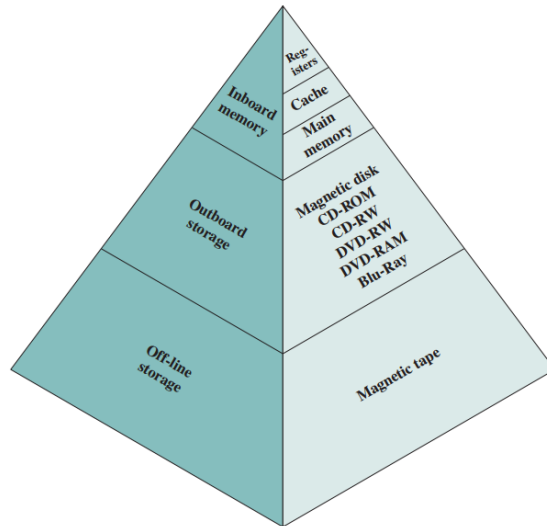
- Dung lượng lớn hơn, thời gian truy xuất chậm hơn.
- ❖ Giải pháp để giải quyết vấn đề về bộ nhớ thì không dựa vào một kiểu bộ nhớ hay công nghệ nào, mà áp dụng sự phân cấp bộ nhớ (*memory hierarchy*).
- ❖ Sự phân cấp tiêu biểu được minh họa trong Hình 5.1. Xét từ trên xuống:
 - Giảm giá thành mỗi bit nhớ.
 - Tăng dung lượng.
 - Tăng thời gian truy xuất.
 - Giảm tần suất truy xuất bộ nhớ.

ThS. GVC Tô Oai Hùng

4

4

Sự Phân Cấp Bộ Nhớ



Hình 5.1: Sự phân cấp bộ nhớ.

ThS. GVC Tô Oai Hùng

5

5

5.2 BỘ NHỚ TRONG

ThS. GVC Tô Oai Hùng

6

6

5.2 Bộ Nhớ Trong

1. Bộ nhớ cache.
2. Bit nhớ.
3. Tổ chức bộ nhớ.
4. Phân loại bộ nhớ

ThS. GVC Tô Oai Hùng

7

7

5.2.1 Bộ Nhớ Cache

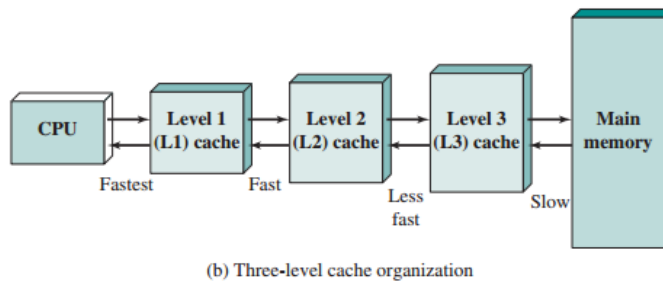
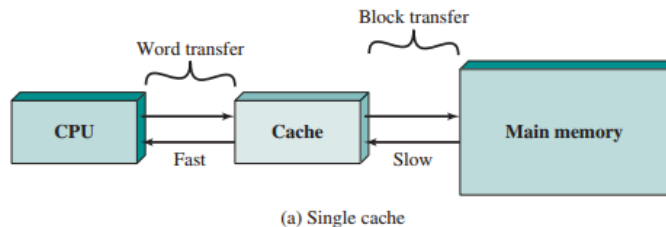
- ❖ Bộ nhớ cache (*bộ nhớ truy cập nhanh/bộ nhớ đệm*) được thiết kế nhằm kết hợp bộ nhớ có dung lượng nhỏ, tốc độ truy xuất nhanh, đắt tiền với bộ nhớ có dung lượng lớn, tốc độ truy xuất chậm, ít đắt tiền. Hình 5.2 (a).
- ❖ Khi bộ xử lý muốn đọc một từ của bộ nhớ chính, nó kiểm tra từ đó có trong cache không. Nếu có thì từ đó được chuyển đến bộ xử lý. Nếu không, một khối của bộ nhớ chính được đọc vào trong cache và từ cần đọc sẽ được chuyển đến bộ xử lý.

ThS. GVC Tô Oai Hùng

8

8

Bộ Nhớ Cache



Hình 5.2: Bộ nhớ cache và bộ nhớ chính.

ThS. GVC Tô Oai Hùng

9

9

Bộ Nhớ Cache

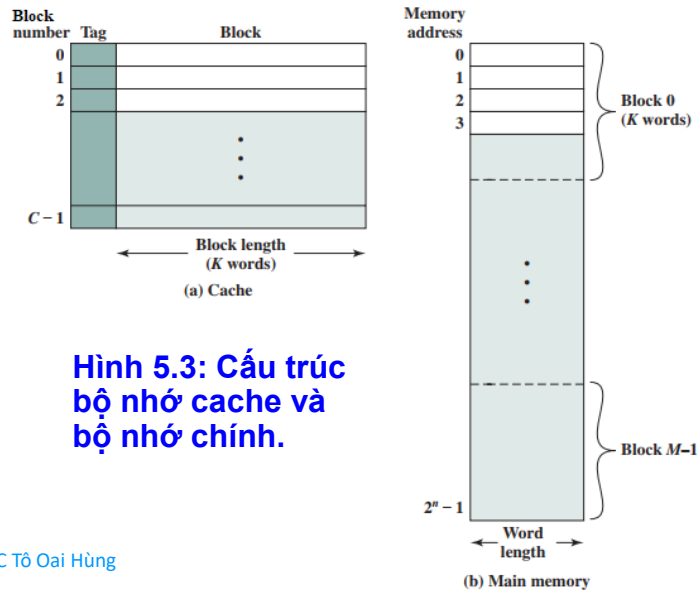
- ❖ Hình 5.2 (b) mô tả việc sử dụng nhiều mức cache. Cache L2 thì chậm hơn và lớn hơn cache L1. Cache L3 thì chậm hơn và lớn hơn cache L2.
- ❖ Hình 5.3 mô tả cấu trúc của bộ nhớ cache và bộ nhớ chính.
- ❖ Bộ nhớ chính có M ($M = 2^n / K$) khối (*block*), mỗi khối có độ dài cố định là K từ.
- ❖ Bộ nhớ cache có C khối. Mỗi khối chứa K từ và một vài bit điều khiển trong của thẻ (*tag*).
- ❖ Kích thước mỗi khối trong cache có thể là 32 bit.

ThS. GVC Tô Oai Hùng

10

10

Bộ Nhớ Cache



Hình 5.3: Cấu trúc bộ nhớ cache và bộ nhớ chính.

ThS. GVC Tô Oai Hùng

11

11

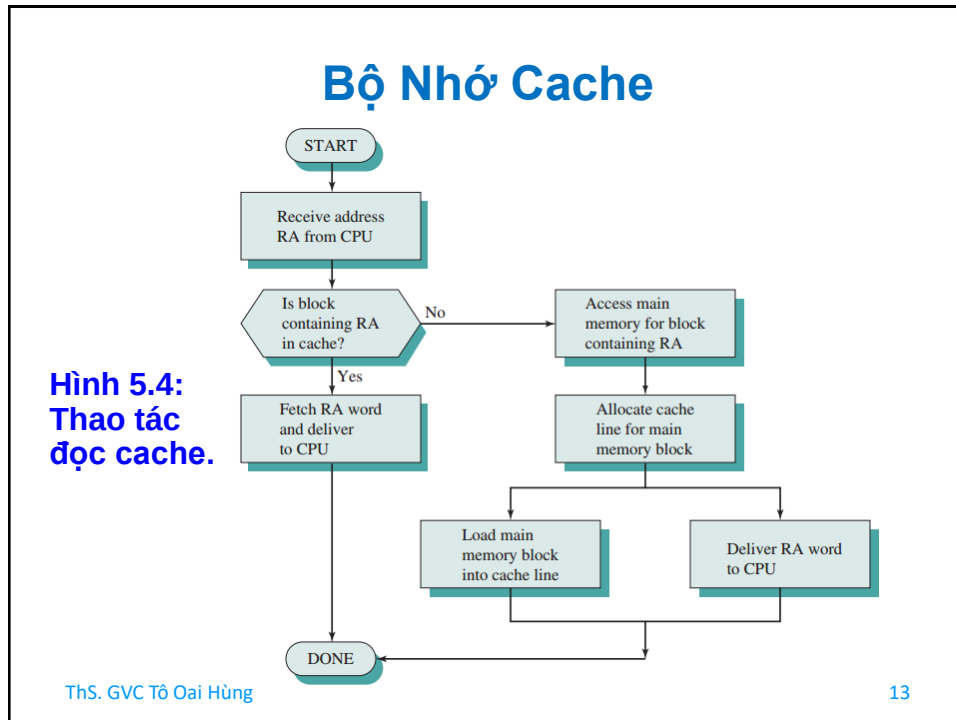
Bộ Nhớ Cache

- ❖ Hình 5.4 minh họa thao tác đọc bộ nhớ cache.
- ❖ Bộ xử lý đưa ra địa chỉ đọc (read address - RA) của từ cần đọc. Nếu từ đó có chứa trong cache, nó sẽ được đưa đến bộ xử lý.
- ❖ Ngược lại, khối chứa từ đó được nạp vào cache và từ đó được đưa đến bộ xử lý.

ThS. GVC Tô Oai Hùng

12

12



13

5.2.2 Bit Nhớ

- ❖ Thành phần cơ bản của bộ nhớ bán dẫn (*semiconductor memory*) được sử dụng làm bộ nhớ chính (*main memory*) là các ô nhớ (*memory cell*).
- ❖ Mỗi ô nhớ có một địa chỉ riêng và là đơn vị hoạt động của bộ nhớ.
- ❖ Thông thường, mỗi ô nhớ là 1 byte.
- ❖ Hầu hết, các ô nhớ bán dẫn có một số đặc điểm sau:
 - Có hai trạng thái (0 và 1).
 - Có khả năng được ghi vào (thiết lập trạng thái).
 - Có khả năng được đọc ra (nhận biết trạng thái).

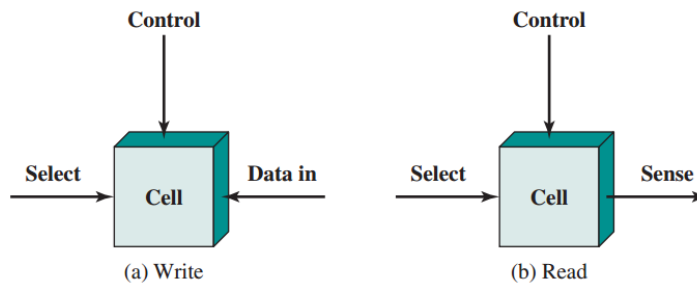
ThS. GVC Tô Oai Hùng

14

14

Bit Nhớ

❖ Hình 5.4 mô tả hoạt động của một ô nhớ.



Hình 5.5: Hoạt động của ô nhớ.

ThS. GVC Tô Oai Hùng

15

15

5.2.3 Tổ Chức Bộ Nhớ

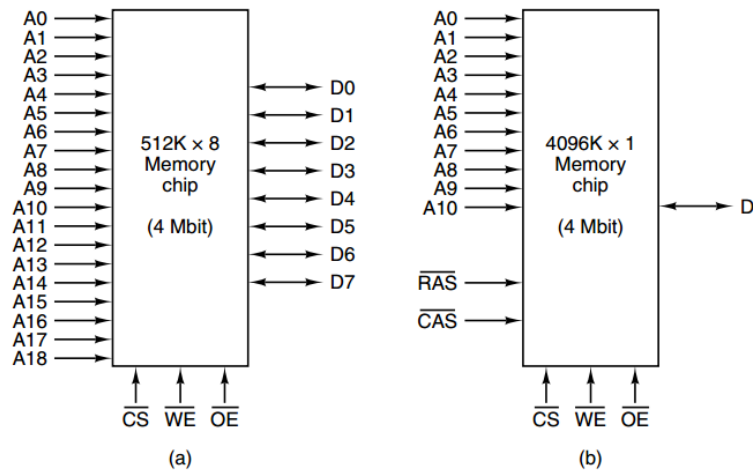
- ❖ Với kích thước bộ nhớ cho trước, có nhiều cách để tổ chức vi mạch (IC/chip) nhớ.
- ❖ Hình 5.6 cho thấy hai tổ chức có thể của vi mạch nhớ trước đây có kích thước là 4 Mbit: 512 K × 8 bit và 4096 K × 1 bit.
- ❖ Lưu ý:
 - /CS hoặc /CE (*chip select/chip enable*).
 - /OE hoặc /RD (*output enable/read* - cho phép đọc).
 - /WE (*write enable* - cho phép ghi).

ThS. GVC Tô Oai Hùng

16

16

Tổ Chức Bộ Nhớ



Hình 5.6: Hai cách tổ chức chip nhớ 4 Mbit.

ThS. GVC Tô Oai Hùng

17

17

5.2.4 Phân Loại Bộ Nhớ

- ❖ Bộ nhớ trong thường được chia thành 2 loại: Bộ nhớ chính (RAM, ROM) và bộ nhớ đệm (cache).
- ❖ Thuật ngữ bộ nhớ truy xuất ngẫu nhiên (*random-access memory* - RAM) có thể bị lạm dụng.
- ❖ Bởi vì tất cả các chip nhớ đều có thể truy xuất ngẫu nhiên.
- ❖ Tuy nhiên, từ RAM đã được sử dụng quen thuộc.
- ❖ RAM được chia làm hai loại: RAM tĩnh (*static RAM* – SRAM) và RAM động (*dynamic RAM* - DRAM).
- ❖ Đặc điểm của bộ nhớ RAM là có thể đọc/ghi dữ liệu vào bộ nhớ dễ dàng.

ThS. GVC Tô Oai Hùng

18

18

SRAM

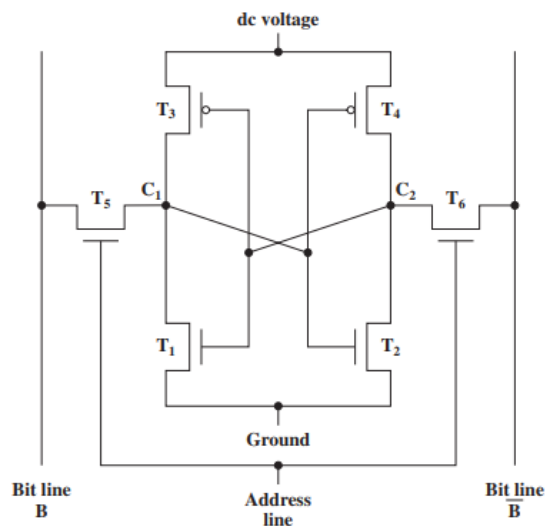
- ❖ RAM tĩnh (SRAM) được tạo bởi các mạch tương tự như flip-flop D.
- ❖ Loại bộ nhớ này có đặc tính là nội dung của nó được duy trì miễn chừng nào còn nguồn điện cung cấp.
- ❖ Thời gian truy xuất tính bằng nano giây hoặc ít hơn. Vì lý do này, SRAM thường dùng làm bộ nhớ cache.
- ❖ Hình 5.7 là cấu trúc SRAM tiêu biểu cho một ô nhớ.

ThS. GVC Tô Oai Hùng

19

19

SRAM



ThS. GVC Tô Oai Hùng

Hình 5.7: Ô nhớ của SRAM.

20

20

DRAM

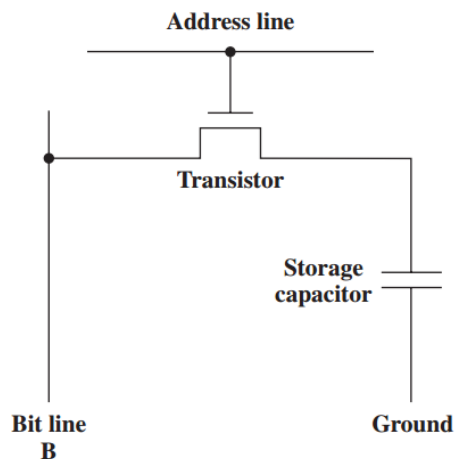
- ❖ RAM động (DRAM) được tạo từ một dãy các ô, mỗi ô chứa một transistor và một tụ điện.
- ❖ Các tụ điện có thể được nạp và xả, sự có hay không có điện tích trong tụ sẽ được biểu thị bằng giá trị nhị phân 1 hoặc 0.
- ❖ Vì điện tích trong tụ bị rò rỉ, nên mỗi bit trong DRAM phải được làm tươi (*refresh*).
- ❖ Do cần có mạch làm tươi, nên cấu trúc DRAM phức tạp hơn SRAM.
- ❖ Dung lượng của DRAM lớn hơn SRAM.
- ❖ Hình 5.8 là một cấu trúc tiêu biểu của DRAM cho 1 bit nhớ.

ThS. GVC Tô Oai Hùng

21

21

DRAM



Hình 5.8: Ô nhớ của DRAM.

ThS. GVC Tô Oai Hùng

22

22

DRAM

- ❖ Có nhiều kiểu chip RAM động:
 - SDRAM (*synchronous DRAM* – RAM động đồng bộ).
 - DDR SDRAM (*double data rate SDRAM*).
 - RDRAM (*Rambus dynamic RAM*).
- ❖ Hình 5.9 cho thấy ảnh thật của một loại DRAM kiểu DIMM (*dual inline memory module* - bộ nhớ hai hàng chân).



Hình 5.9:
DIMM.

DIMM

23

23

ROM

- ❖ ROM (*Read Only Memory* – Bộ nhớ chỉ đọc), là loại bộ nhớ không thể thay đổi hay xóa có chủ đích.
- ❖ Dữ liệu trong ROM được thêm vào trong lúc sản xuất ra nó, bằng cách là thông qua vật liệu cảm quang của mặt nạ để khắc lên trên bề mặt ROM các bit của chương trình.
- ❖ ROM thì rẻ hơn RAM nhiều. Tuy nhiên, nó không linh hoạt, bởi vì nó không thể thay đổi sau khi sản xuất.
- ❖ Tuy nhiên, các loại ROM sau này có thể xóa và ghi lại nhiều lần.

24

ROM

❖ Một số loại ROM khác:

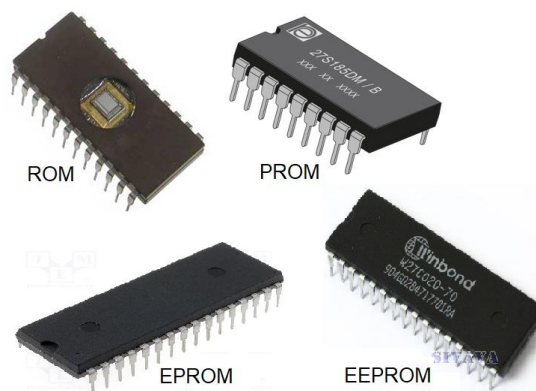
- PROM (programmable ROM – ROM lập trình được): Có thể được lập trình 1 lần bằng thiết bị chuyên dụng.
- EPROM (erasable PROM – ROM có thể lập trình và xoá): Không chỉ lập trình được mà còn xoá được bằng thiết bị chuyên dụng.
- EEPROM (electrically EPROM – ROM có thể lập trình và xoá bằng điện): Có thể xoá từng byte dữ liệu bằng các xung điện (bằng phần mềm).
- Hình 5.10 cho thấy một số loại ROM.

ThS. GVC Tô Oai Hùng

25

25

ROM



Hình 5.10: Một số loại bộ nhớ ROM.

ThS. GVC Tô Oai Hùng

26

26

5.3. Vấn Đề Quản Lý Bộ Nhớ

- ❖ Bộ nhớ chính là nơi mà CPU có thể truy cập trực tiếp.
- ❖ Dữ liệu của chương trình phải được ghi vào bộ nhớ chính trước khi được xử lý.
- ❖ Việc trao đổi thông tin giữa CPU với môi trường ngoài thông qua thao tác đọc/ghi dữ liệu vào một địa chỉ trong bộ nhớ.
- ❖ Bộ nhớ chính được tổ chức như mảng một chiều các từ nhớ (word), mỗi từ nhớ có một địa chỉ.

ThS. GVC Tô Oai Hùng

27

27

Vấn Đề Quản Lý Bộ Nhớ

- ❖ Trong hệ thống đơn chương, bộ nhớ chính được chia thành hai phần:
 - Một phần dành cho hệ điều hành.
 - Một phần dành cho các chương trình đang được thực thi.
- ❖ Trong một hệ thống đa chương:
 - Phân bộ nhớ dành cho các chương trình người dùng phải được chia nhỏ hơn.
- ❖ Nhiệm vụ phân chia bộ nhớ được thực hiện động bởi hệ điều hành gọi là quản lý bộ nhớ.

ThS. GVC Tô Oai Hùng

28

28

Vấn Đề Quản Lý Bộ Nhớ

The screenshot shows the Windows Task Manager Performance tab. The 'Memory' section is highlighted with a red box, indicating 39% usage. Below it, a list of processes is shown with their respective CPU and memory usage.

Name	CPU	Memory
Apps (5)		
Google Chrome (8)	1.4%	205.3 MB
Microsoft PowerPoint (2)	0%	144.5 MB
Background processes (64)		
Adobe Acrobat Update Service (...)	0%	0.4 MB
Alps Pointing-device Driver	0%	0.8 MB
Alps Pointing-device Driver	0%	3.8 MB
Alps Pointing-device Driver for ...	0%	1.1 MB
Antimalware Service Executable	0%	82.4 MB
ApMsgFwd	0%	1.1 MB
Bluetooth Radio Management S...	0%	0.5 MB

ThS. GVC Tô Oai Hùng

29

29

Vấn Đề Quản Lý Bộ Nhớ

- ❖ Quản lý bộ nhớ là công việc của hệ điều hành cùng với sự hỗ trợ của phần cứng nhằm phân phối, sắp xếp các tiến trình trong bộ nhớ sao cho hiệu quả.
- ❖ Ba chức năng chính của HĐH đối với quản lý bộ nhớ:
 - Theo dõi những phần nào của bộ nhớ đang được sử dụng.
 - Quyết định những tiến trình nào sẽ được nạp vào bộ nhớ.
 - Định vị và tái định vị không gian bộ nhớ khi cần thiết.
- ❖ Mục tiêu quản lý bộ nhớ: nạp càng nhiều tiến trình vào bộ nhớ càng tốt (gia tăng mức độ đa chương).

ThS. GVC Tô Oai Hùng

30

30

Vấn Đề Quản Lý Bộ Nhớ

- ❖ Các yêu cầu đối với việc quản lý bộ nhớ.
 - Cấp phát bộ nhớ cho các tiến trình.
 - Tái định vị (*relocation*): Khi swapping, ...
 - Bảo vệ: Phải kiểm tra truy xuất bộ nhớ có hợp lệ không.
 - Chia sẻ: Cho phép các tiến trình chia sẻ vùng nhớ chung.
 - Kết gán địa chỉ nhớ luận lý của user vào địa chỉ thực.
- ❖ Hàng đợi nhập hệ thống: Các chương trình trên đĩa đang chờ được nạp vào bộ nhớ.

ThS. GVC Tô Oai Hùng

31

31

Vấn Đề Quản Lý Bộ Nhớ

- ❖ Địa chỉ vật lý (*physical address* - địa chỉ thực): Là một vị trí thực trong bộ nhớ chính.
- ❖ Địa chỉ luận lý / ảo (*logical address*): Địa chỉ được CPU tạo ra khi thực thi, là một vị trí nhớ được diễn tả trong một chương trình.
 - Các trình biên dịch (*compiler*) tạo ra mã lệnh chương trình mà trong đó mọi tham chiếu bộ nhớ đều là địa chỉ luận lý.
 - Địa chỉ tuyệt đối (*absolute address*): Địa chỉ "tương đương" với địa chỉ thực.
 - Địa chỉ tương đối (*relative address*) (khả tái định vị - *relocatable address*): Địa chỉ tính từ một điểm cơ sở (*base address*)

ThS. GVC Tô Oai Hùng

32

32

Vấn Đề Quản Lý Bộ Nhớ
 trong không gian bộ nhớ mà tiến trình được
 cấp.

**Địa chỉ tuyệt đối = Base address +
 Địa chỉ tương đối**

ThS. GVC Tô Oai Hùng

33

33

Vấn Đề Quản Lý Bộ Nhớ

❖ Chuyển đổi địa chỉ:

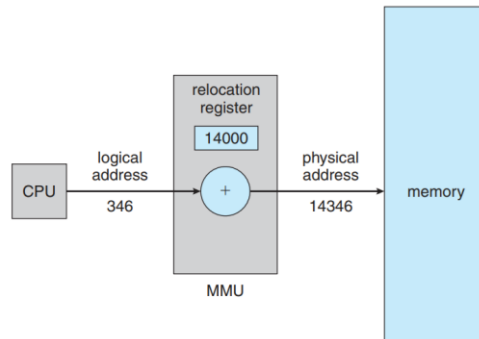
- Khi một lệnh được thực thi, các tham chiếu đến địa chỉ luận lý phải được chuyển đổi thành địa chỉ thực.
- MMU (*Memory Management Unit*): Cơ chế phần cứng được sử dụng để chuyển đổi địa chỉ luận lý thành địa chỉ vật lý tại thời điểm xử lý.

ThS. GVC Tô Oai Hùng

34

34

Vấn Đề Quản Lý Bộ Nhớ



Dynamic relocation using a relocation register.



ThS. GVC Tô Oai Hùng

35

35

Vấn Đề Quản Lý Bộ Nhớ

❖ Có thể thực hiện kết buộc các chỉ thị và dữ liệu với các địa chỉ bộ nhớ vào một trong những thời điểm sau:

1. Thời điểm biên dịch:

- Nếu biết trước địa chỉ bộ nhớ của tiến trình thì có thể kết gán địa chỉ tuyệt đối lúc biên dịch.
- Nếu sau đó thay đổi vị trí thường trú chương trình → biên dịch lại.
- Các chương trình có định dạng MS-DOS .COM bị ràng buộc tại thời điểm biên dịch.

2. Thời điểm nạp:

- Tại thời điểm biên dịch, nếu chưa biết tiến

ThS. GVC Tô Oai Hùng

36

36

Vấn Đề Quản Lý Bộ Nhớ

trình sẽ nằm ở đâu trong bộ nhớ thì TBD phải sinh mã khả tái định vị.

- Vào thời điểm nạp, phải chuyển đổi địa chỉ khả tái định vị thành địa chỉ thực dựa trên địa chỉ nền (*base address*).
- Khi có sự thay đổi địa chỉ nền, chỉ cần nạp lại chương trình để tính toán lại các địa chỉ tuyệt đối, mà không cần biên dịch lại.

3. Thời điểm thực thi:

- Trong quá trình thực thi, tiến trình có thể được di chuyển từ segment này sang segment khác trong bộ nhớ thì quá trình

ThS. GVC Tô Oai Hùng

37

37

Vấn Đề Quản Lý Bộ Nhớ

gán kết địa chỉ được thực hiện tại thời điểm thực thi.

- CPU tạo ra địa chỉ luận lý cho process.
- Cần sự hỗ trợ của phần cứng cho việc ánh xạ địa chỉ:
 - Ví dụ: trường hợp địa chỉ luận lý là relocatable thì có thể dùng thanh ghi base và limit, ...
- Sử dụng các cơ chế swapping, paging, segmentation.
- Được sử dụng trong hầu hết các hệ điều hành.

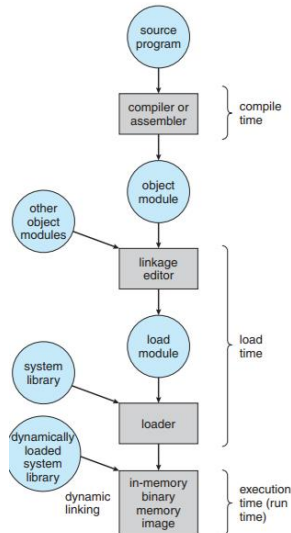
ThS. GVC Tô Oai Hùng

38

38

Vấn Đề Quản Lý Bộ Nhớ

❖ Các bước thực thi chương trình:



**Multistep processing
of a user program.**

ThS. GVC Tô Oai Hùng

39

39

Vấn Đề Quản Lý Bộ Nhớ

❖ Phân mảnh nội (*Internal Fragmentation*):

- Được tạo ra trong phân vùng cố định.
- Mỗi block được cấp phát lớn hơn yêu cầu bộ nhớ một ít.

❖ Phân mảnh ngoại vi (*External Fragmentation*):

- Được tạo ra trong phân vùng động.
- Tổng bộ nhớ trống thỏa yêu cầu, nhưng không liên tục.

❖ Giải pháp phân mảnh ngoại vi:

- Chuyển các vùng trống thành một khối bộ nhớ liên tục.
- Chỉ thực hiện được nếu HĐH hỗ trợ biên dịch địa chỉ trong thời gian thực thi.

ThS. GVC Tô Oai Hùng

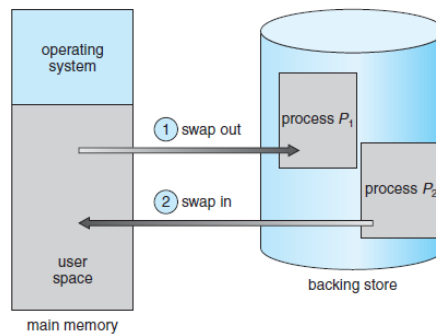
40

40

Vấn Đề Quản Lý Bộ Nhớ

❖ Swapping:

- Tiến trình ở trạng thái chờ được tạm thời chuyển ra bộ nhớ phụ (swap out), sau đó được chuyển vào bộ nhớ chính (swap in) để tiếp tục xử lý.



ThS. GVC Tô Oai Hùng

main memory
Swapping of two processes using a disk as a backing store.

41

41

Vấn Đề Quản Lý Bộ Nhớ

- Tăng số lượng các tiến trình đồng hành → hiệu quả trong môi trường đa chương.
- Cần xác định vị trí của tiến trình trong bộ nhớ chính khi được chuyển ra/vào, phụ thuộc vào thời điểm kết gán địa chỉ.
 - Thời điểm nạp: Cần phải cấp phát lại đúng vùng nhớ cho tiến trình.
 - Thời điểm xử lý: Có thể nạp tiến trình vào một vùng nhớ bất kỳ.
- Cần sử dụng bộ nhớ phụ.
- Thời gian chuyển đổi: Mỗi tiến trình cần được phân chia thời gian sử dụng CPU sao cho không cảm thấy bị chậm trễ do swap.

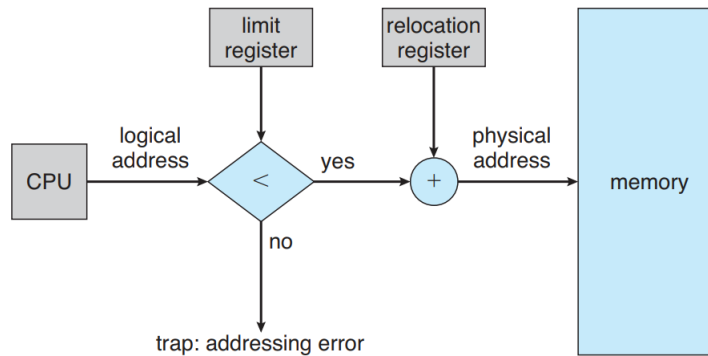
ThS. GVC Tô Oai Hùng

42

42

Vấn Đề Quản Lý Bộ Nhớ

- ❖ Bảo vệ bộ nhớ (Memory Protection):
 - Ngăn một tiến trình truy cập bộ nhớ không hợp lệ.



Hardware support for relocation and limit registers.

ThS. GVC Tô Oai Hùng

43

43

Vấn Đề Quản Lý Bộ Nhớ

- Cần có hỗ trợ của phần cứng, sử dụng hai thanh ghi:
 - Relocation register: Ghi vị trí bắt đầu của tiến trình trong bộ nhớ vật lý.
 - Limit register: Ghi phạm vi địa chỉ logic.

ThS. GVC Tô Oai Hùng

44

44

5.4 Mô Hình Quản Lý Bộ Nhớ Thực

- ❖ Cấp phát liên tục.
 - Phân vùng cố định.
 - Phân vùng động.
- ❖ Cấp phát không liên tục.
 - Phân trang.
 - Phân đoạn.

ThS. GVC Tô Oai Hùng

45

45

5.4.1 Cấp Phát Liên Tục

1. Phân vùng cố định.
2. Phân vùng động.

ThS. GVC Tô Oai Hùng

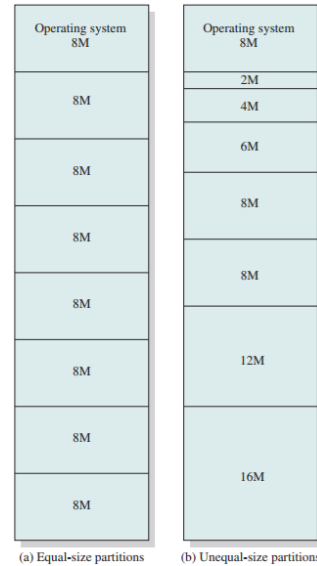
46

46

Phân Vùng Cố Định

- ❖ Hệ điều hành chiếm một phần của bộ nhớ chính và phần còn lại được sử dụng cho các tiến trình.
- ❖ Phần còn lại có thể được giả định phân chia thành n phân vùng (*partition*) có kích thước cố định (bằng nhau hoặc không bằng nhau).
- ❖ Các tiến trình có nhu cầu bộ nhớ sẽ được lưu trữ trong hàng đợi.
- ❖ Có hai cách tổ chức:
 - Sử dụng nhiều hàng đợi.
 - Sử dụng một hàng đợi.

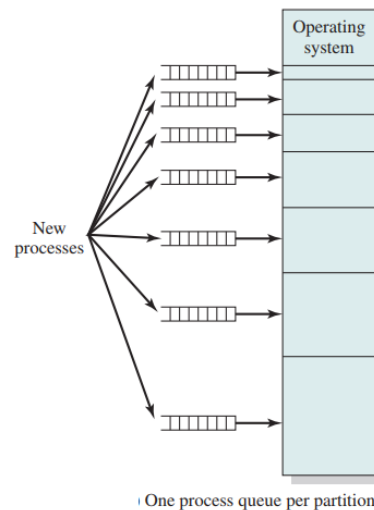
ThS. GVC Tô Oai Hùng



47

Phân Vùng Cố Định

- ❖ Sử dụng nhiều hàng đợi:
 - Mỗi phân vùng có một hàng đợi tương ứng.
 - Tiến trình được tạo sẽ được đưa vào hàng đợi của phân vùng có kích thước nhỏ nhất đủ chứa nó.
 - Hạn chế: Có thể có một số hàng đợi trống vì không có tiến trình có kích thước phù hợp.



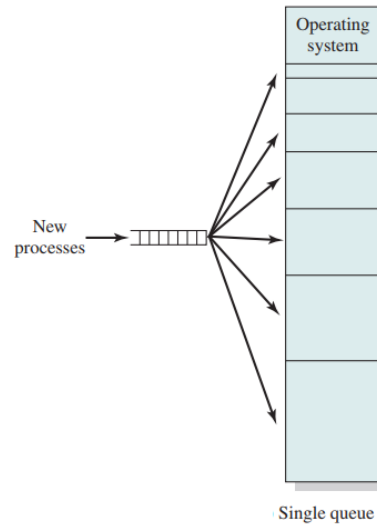
ThS. GVC Tô Oai Hùng

48

48

Phân Vùng Cố Định

- ❖ Sử dụng một hàng đợi:
 - Chỉ có một hàng đợi chung cho tất cả partition.
 - Khi cần nạp một tiến trình vào bộ nhớ chính → chọn partition nhỏ nhất còn trống.



ThS. GVC Tô Oai Hùng

49

49

Phân Vùng Cố Định

- ❖ Nhận xét sử dụng một hàng đợi:
 - Đơn giản, chi phí thấp.
 - Kích thước tiến trình không vừa đúng bằng kích thước phân vùng chứa nó, phần bộ nhớ còn lại sẽ lãng phí (phân mảnh nội).
 - Số lượng phân vùng được cài đặt cố định → ảnh hưởng đến mức độ đa chương.
 - Trong môi trường đa chương cần tái định vị (*relocation*) và bảo vệ các tiến trình.

ThS. GVC Tô Oai Hùng

50

50

Phân Vùng Cố Định

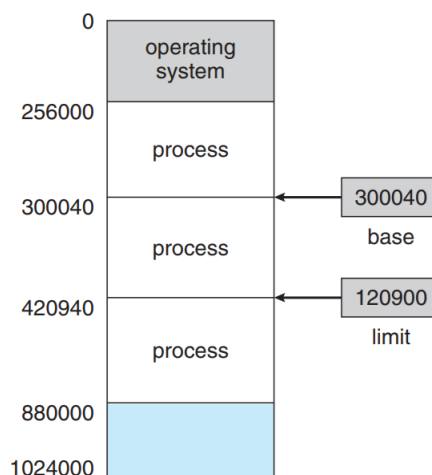
- ❖ Tái định vị vào thời điểm nạp chương trình: cập nhật lại các địa chỉ trong chương trình khi chương trình được nạp vào bộ nhớ.
- ❖ Bảo vệ: Sử dụng hai thanh ghi.
 - Thanh ghi nền (base register).
 - Thanh ghi giới hạn (limit register).
 - Khi tiến trình được tạo:
 - Nạp vào thanh ghi nền địa chỉ bắt đầu của phân vùng được cấp cho tiến trình.
 - Nạp vào thanh ghi giới hạn kích thước của tiến trình.

ThS. GVC Tô Oai Hùng

51

51

Phân Vùng Cố Định



ThS. GVC Tô Oai Hùng

A base and a limit register define a logical address space.

52

52

Phân Vùng Động

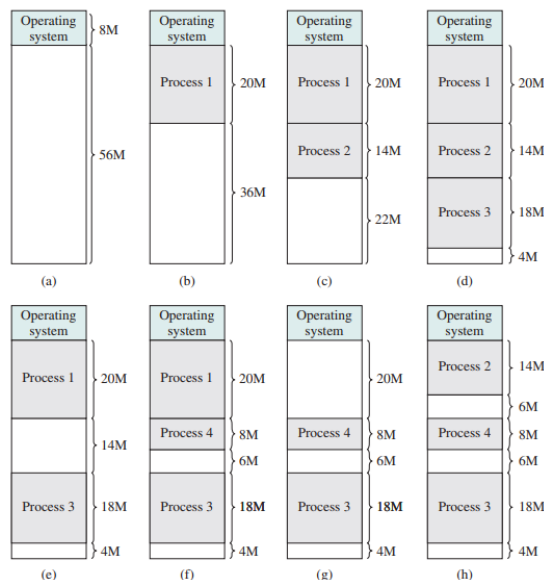
- ❖ Cấp phát vùng nhớ vừa đúng với kích thước của tiến trình được tạo.
- ❖ Thu hồi vùng nhớ khi tiến trình kết thúc.
- ❖ Kích thước vùng nhớ cấp cho tiến trình và vị trí bắt đầu của các phân vùng là động.

ThS. GVC Tô Oai Hùng

53

53

Phân Vùng Động



**The Effect of
Dynamic
Partitioning.**

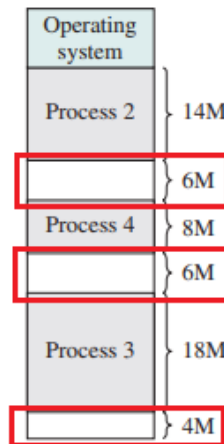
ThS. GVC Tô Oai Hùng

54

54

Phân Vùng Động

- ❖ Nhận xét: Không phân mảnh nội, nhưng phân mảnh ngoại vi.
 - Ví dụ: Tiến trình mới có kích thước 7M?



ThS. GVC Tô Oai Hùng

55

55

Phân Vùng Động

- ❖ Quản lý không gian bộ nhớ: Có hai phương pháp.
 - Sử dụng mảng bit: Bộ nhớ được chia thành những đơn vị cấp phát, mỗi bit trên dãy bit cho biết trạng thái của một đơn vị
 - 0: Còn trống.
 - 1: Đã cấp phát cho tiến trình.
 - Sử dụng danh sách liên kết: Mỗi phần tử quản lý một vùng trên bộ nhớ, gồm ba trường chính:
 - Trường thứ nhất cho biết khối nhớ đã cấp phát (P: *process*) hay đang còn trống (H: *hole*).

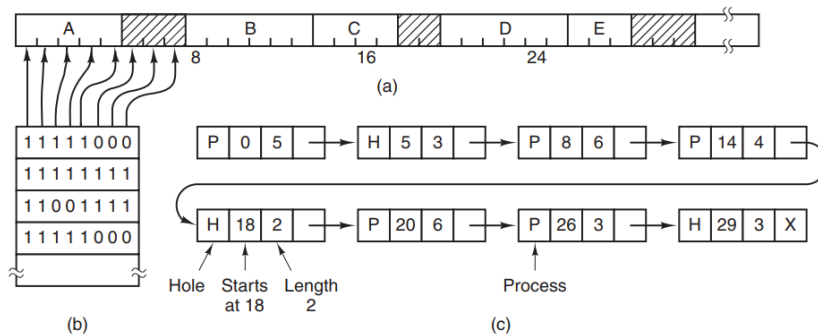
ThS. GVC Tô Oai Hùng

56

56

Phân Vùng Động

- Trường thứ hai cho biết thứ tự của đơn vị cấp phát đầu tiên trong block (*start*)
- Trường thứ ba cho biết block gồm bao nhiêu đơn vị cấp phát (*length*).



(a) A part of memory with five processes and three holes. The tick marks show the memory allocation units. The shaded regions (0 in the bitmap) are free. (b) The corresponding bitmap. (c) The same information as a list.

7

57

Phân Vùng Động

❖ Các thuật toán cấp phát vùng nhớ cho tiến trình:

- **Best-fit:** Chọn khối nhớ có kích thước nhỏ nhất lớn hơn hoặc bằng kích thước của tiến trình cần được nạp vào bộ nhớ.
- **First-fit:** Chọn khối nhớ trống đầu tiên có kích thước đủ lớn để nạp tiến trình.
- **Last-fit:** Chọn khối nhớ trống cuối cùng có kích thước đủ lớn để nạp tiến trình.
- **Next-fit:** Tương tự như First-fit nhưng HĐH bắt đầu quét từ khối nhớ trống kế sau khối vừa được cấp phát và chọn khối nhớ trống kế tiếp đủ lớn để nạp tiến trình.
- **Worst-fit:** Chọn khối nhớ có kích thước lớn nhất để nạp tiến trình.

ThS. GVC Tô Văn Hùng

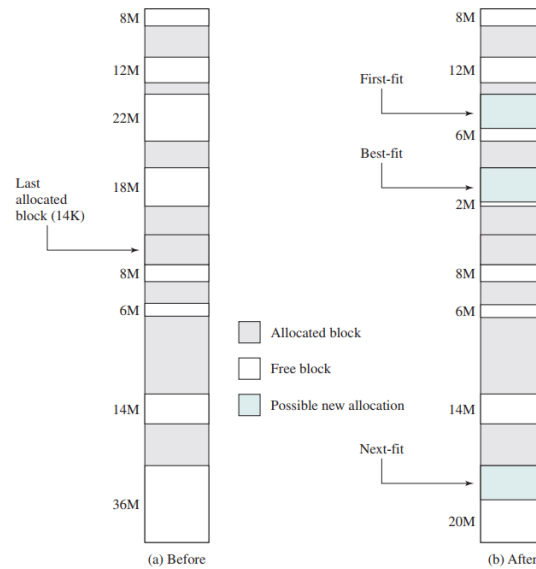
58

58

Phân Vùng Động

❖ Ví dụ: Cấp phát khối nhớ cho tiến trình có kích thước 16M theo các thuật toán:

- First-fit.
- Best-fit.
- Next-fit.
- Last-fit.
- Worst-fit.



ThS. GVC Tô Oai Hùng

59

5.4.2 Cấp Phát Không Liên Tục

1. Phân trang.
2. Phân đoạn.

ThS. GVC Tô Oai Hùng

60

60

Phân Trang

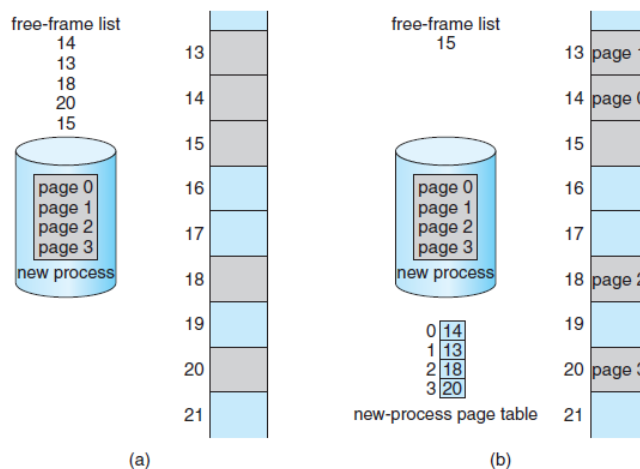
- ❖ Cơ chế phân trang (*Paging*) cho phép không gian địa chỉ thực (*Physical address space*) của một process có thể không liên tục nhau.
- ❖ Bộ nhớ vật lý được chia thành các khối (*block*) có kích thước cố định và bằng nhau, gọi là khung trang (*page frame*), kích thước của frame là lũy thừa của 2 (512 bytes - 16Mb).
- ❖ Bộ nhớ luận lý (không gian địa chỉ luận lý - *Logical address space*) cũng được chia thành các khối có cùng kích thước với khung trang, gọi là trang (*page*).
- ❖ Khi cần nạp một tiến trình, các trang của tiến trình sẽ được nạp vào các khung trang còn trống.

ThS. GVC Tô Oai Hùng

61

61

Phân Trang



Free frames (a) before allocation and (b) after allocation.

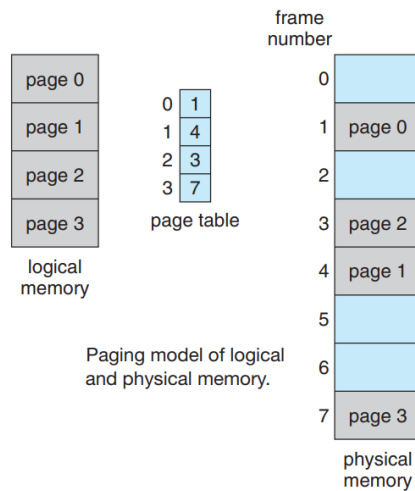
ThS. GVC Tô Oai Hùng

62

62

Phân Trang

- ❖ Hệ điều hành phải thiết lập một bảng trang (*page table*) để ánh xạ địa chỉ luận lý thành địa chỉ thực.
- ❖ Mỗi tiến trình có một bảng trang, được quản lý bằng một con trỏ lưu giữ trong PCB.
- ❖ Mỗi phần tử trong *page table* chứa địa chỉ bắt đầu của vị trí lưu trữ trang tương ứng trong bộ nhớ vật lý.



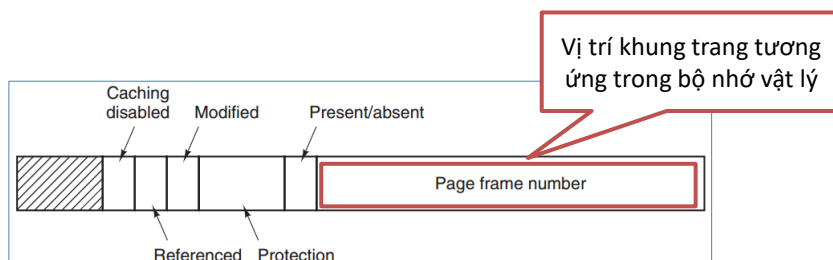
ThS. GVC Tô Oai Hùng

63

63

Phân Trang

- ❖ Cấu trúc một phần tử trong *page table*:

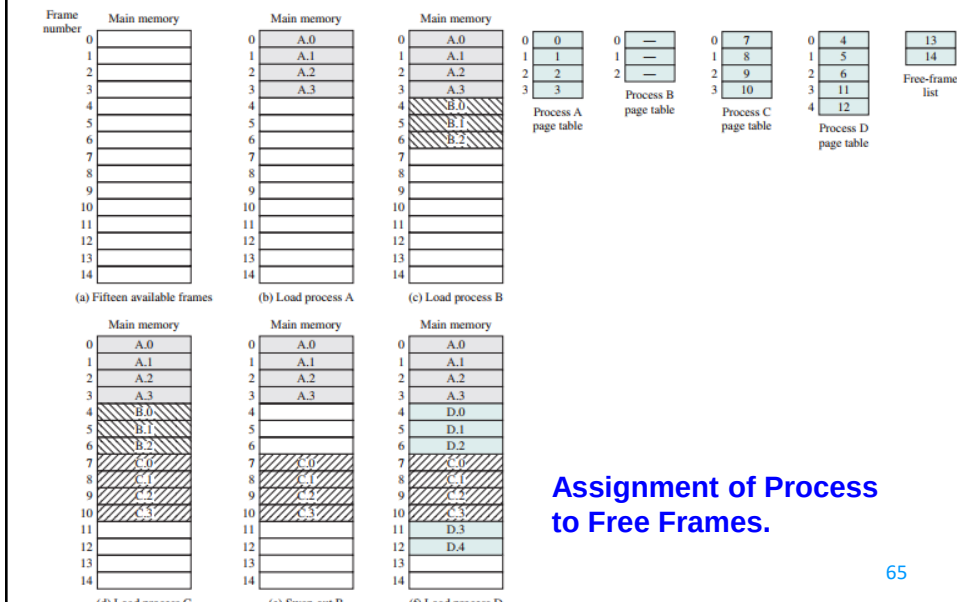


ThS. GVC Tô Oai Hùng

64

64

Phân Trang



65

Phân Trang

- ❖ Địa chỉ logic phát sinh từ CPU được chia thành hai phần:
 - Page number (p): Số hiệu trang, sử dụng như chỉ mục đến phần tử tương ứng trong bảng trang.
 - Page offset (d): Địa chỉ tương đối trong trang, kết hợp với địa chỉ bắt đầu của trang để tạo ra địa chỉ vật lý mà trình quản lý bộ nhớ sử dụng (độ dời trang).
- ❖ Kích thước trang do phân cứng qui định, thường là lũy thừa của 2 (512 byte – 16 MB).
- ❖ Ví dụ: Nếu kích thước của không gian địa chỉ là 2^m và kích thước trang là 2^n :

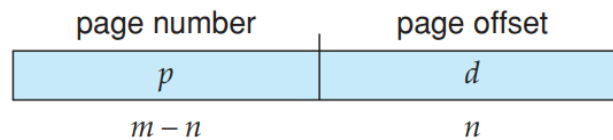
ThS. GVC Tô Oai Hùng

66

66

Phân Trang

- p: m - n bit cao của địa chỉ logic.
- d: n bit thấp của địa chỉ logic.



- bảng trang sẽ có tổng cộng $2^m/2^n = 2^{m-n}$ mục (entry).

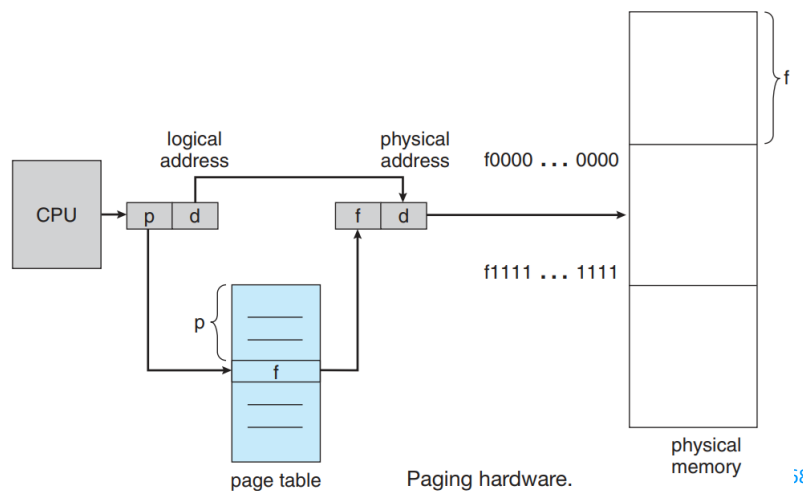
ThS. GVC Tô Oai Hùng

67

67

Phân Trang

- ❖ Phần cứng chuyển đổi địa chỉ logic sang địa chỉ vật lý:



68

Phân Trang

- ❖ Quá trình chuyển đổi địa chỉ logic sang địa chỉ vật lý:
 - Lấy m bit cao của địa chỉ => số thứ tự trang (vị trí trang trong bảng trang).
 - Dựa vào phần tử tương ứng trong bảng trang, tìm được số thứ tự khung vật lý k (vị trí khung trang trong bộ nhớ vật lý).
 - Địa chỉ vật lý bắt đầu của khung là $k * 2^n$.
 - Địa chỉ vật lý của byte được tham chiếu là địa chỉ bắt đầu của khung cộng với độ dời.
 - => Chỉ cần thêm số khung vào trước dãy bit biểu diễn độ lệch.

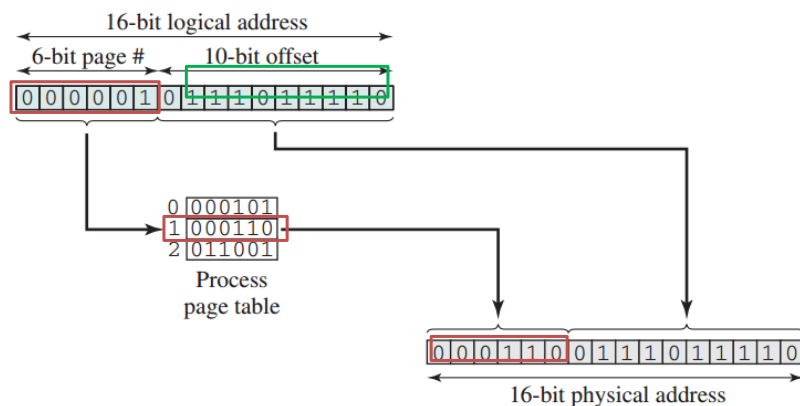
ThS. GVC Tô Oai Hùng

69

69

Phân Trang

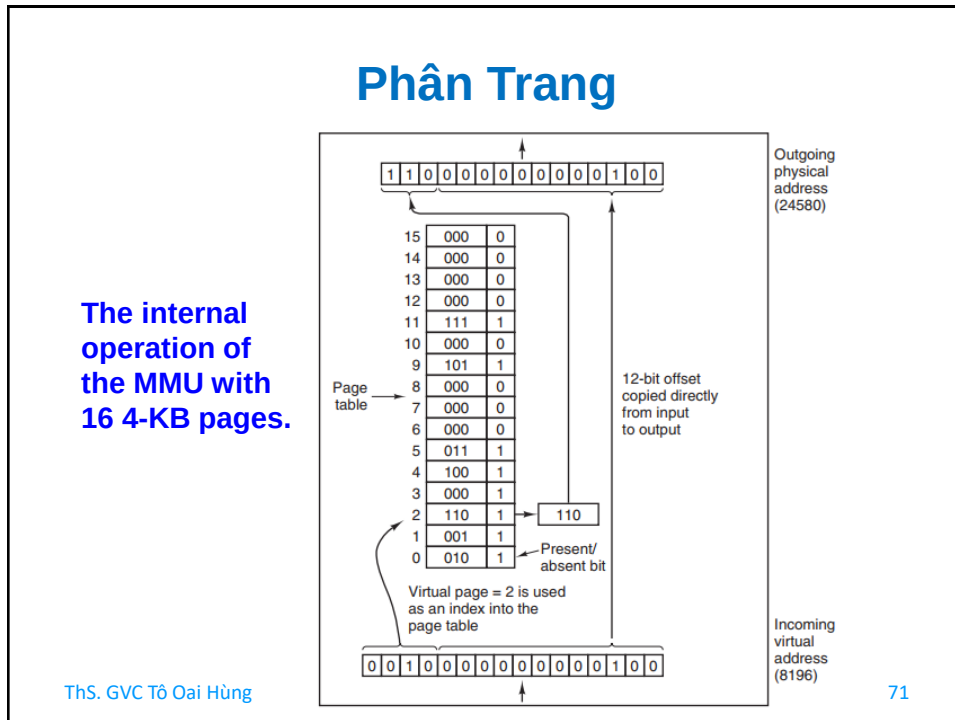
- ❖ Các ví dụ:



ThS. GVC Tô Oai Hùng

70

70



71

Phân Trang

- ❖ Cài đặt bảng trang.
 - Bảng trang thường được lưu giữ trong bộ nhớ chính.
 - Mỗi tiến trình được hệ điều hành cấp một bảng trang.
 - Thanh ghi page-table base (PTBR) trỏ đến bảng trang.
 - Thanh ghi page-table length (PTLR) biểu thị kích thước của bảng trang (có thể được dùng trong cơ chế bảo vệ bộ nhớ)
- ❖ Nhận xét về cơ chế phân trang:
 - Cấp phát không liên tục: Các trang của

ThS. GVC Tô Oai Hùng

72

72

Phân Trang

cùng một tiến trình có thể nằm ở các vị trí không liên tục trong bộ nhớ → không phân mảnh ngoại vi.

- Có phân mảnh nội: Khung cuối cùng có thể không được sử dụng hết kích thước.

Phân Trang

❖ Tăng tốc độ phân trang.

- Mỗi tác vụ truy cập dữ liệu/lệnh cần hai thao tác truy xuất vùng nhớ:
 - Dùng page number (p) làm index để truy cập mục trong bảng trang (nằm trong bộ nhớ chính) để lấy số frame.
 - Dùng page offset (d) để truy xuất dữ liệu/lệnh trong frame.
- → Truy cập bộ nhớ chính hai lần → Truy xuất chậm → HĐH dùng một bộ phận cache phần cứng có tốc độ truy xuất và tìm kiếm cao, gọi là Translation Look-aside Buffer (TLB).

Phân Trang

❖ Translation Lookaside Buffer - TLB:

- Là thanh ghi hỗ trợ tìm kiếm truy xuất dữ liệu với tốc độ rất nhanh, nằm trong MMU.
- Số mục của TLB khoảng 8 – 256 mục.
- Mỗi mục chứa thông tin về một trang (page number, frame number, modified bit, valid bit, protection code (quyền đọc/ghi/thực thi).
- Khi có chuyển ngữ cảnh, TLB bị xóa.
- Khi TLB đầy, một mục trong TLB được chọn để thay thế bằng giải thuật Least recently used (LRU).

ThS. GVC Tô Oai Hùng

75

75

Phân Trang

- Ánh xạ địa chỉ ảo:
 - Nếu page number nằm trong TLB (hit, trúng) → lấy ngay được số frame → tiết kiệm được thời gian truy cập bộ nhớ chính để lấy số frame trong bảng trang.
 - Ngược lại (miss, sai), phải lấy số frame từ bảng trang.

ThS. GVC Tô Oai Hùng

76

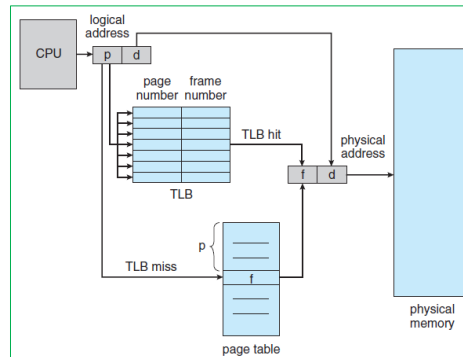
76

Phân Trang

❖ Nội dung một TLB

Valid	Virtual page	Modified	Protection	Page frame
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

Paging hardware với TLB



ThS. GVC Tô Oai Hùng

77

77

Phân Trang

❖ Bảo vệ bộ nhớ:

- Việc bảo vệ bộ nhớ được hiện thực với bit bảo vệ (*protection bit*) trong mỗi phần tử của bảng trang. Các bit này biểu thị các thuộc tính: read-only, read-write, execute-only.
- Ngoài ra, phần tử trong bảng trang còn có một valid bit với hai giá trị:
 - 1: "valid" - cho biết là trang của process, do đó là một trang hợp lệ.
 - 0: "invalid" - cho biết là trang không của process, do đó là một trang bất hợp lệ.
 - Dùng thanh ghi PTLR để kiểm tra truy xuất đến bảng trang có hợp lệ không.

ThS. GVC Tô Oai Hùng

78

78

Phân Trang

❖ Ví dụ:

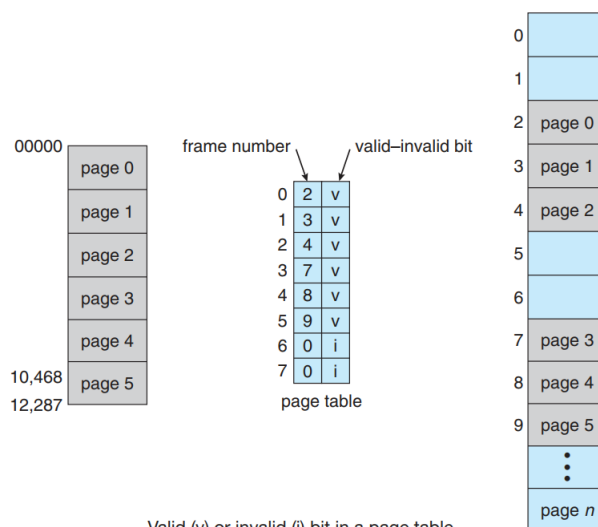
- Hệ thống với không gian địa chỉ 14 bit (0-16383), mỗi trang có kích thước $2K = 2048$.
- Process chỉ sử dụng các địa chỉ từ 0-10468.
- → Địa chỉ trong các trang 0, 1, 2, 3, 4 và 5 được đánh dấu là hợp lệ trong bảng trang
- Các tham chiếu nào ngoài địa chỉ 0-10468 đều là bất hợp lệ.
- Frame 9 (chứa page 5) bị phân mảnh nội, các truy cập vào địa chỉ lên đến 12287 là hợp lệ. Các địa chỉ từ 12288-16383 là không hợp lệ.

ThS. GVC Tô Oai Hùng

79

79

Phân Trang



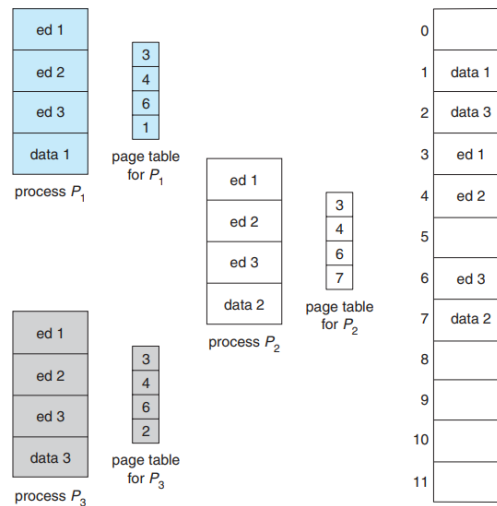
ThS. GVC Tô Oai Hùng

80

80

Phân Trang

- ❖ Chia sẻ các trang nhớ: Cho phép nhiều tiến trình có thể thực thi một mã lệnh tại một thời điểm.



Sharing of code in a paging environment.

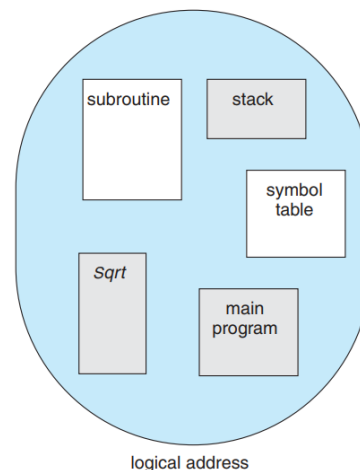
ThS. GVC Tô Oai Hùng

o.i

81

Phân Đoạn (Segmentation)

- ❖ Với góc nhìn của user:
 - Một chương trình người dùng có thể được chia nhỏ, trong đó chương trình và dữ liệu liên quan của nó được chia thành một số phân đoạn.
 - Khi chương trình được biên dịch, trình biên dịch sẽ tạo các đoạn (segment).



Programmer's view of a program.

ThS. GVC Tô Oai Hùng

82

82

Phân Đoạn

- main, procedure, function, local variables, global variables, stack, symbol table, array, ...
- Trình loader sẽ gán mỗi segment một số định danh riêng.

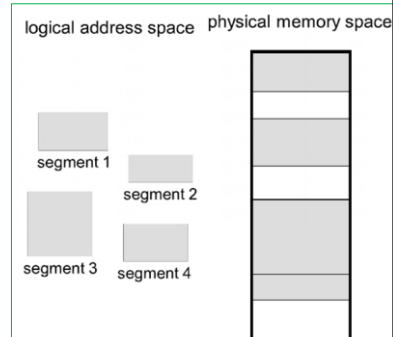
ThS. GVC Tô Oai Hùng

83

83

Phân Đoạn

- ❖ Trong cơ chế phân đoạn, không gian địa chỉ ảo là một tập các segment.
 - Đoạn một dãy các địa chỉ liên tục từ 0.
 - Mỗi phân gồm định danh, kích thước.
 - Kích thước các segment có thể khác nhau, có thể thay đổi khi chương trình thực thi (stack, array, ...).
- ❖ Địa chỉ luận lý bao gồm:
 - Segment number.
 - Offset.



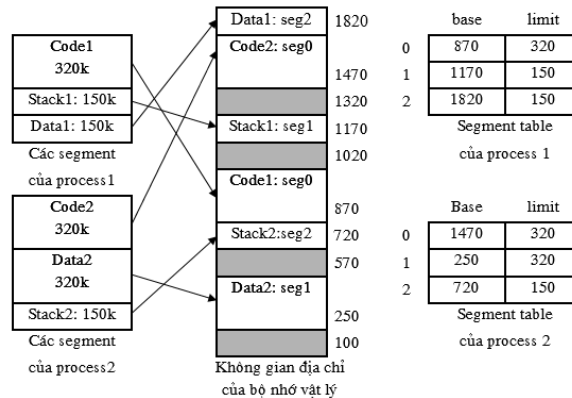
ThS. GVC Tô Oai Hùng

84

84

Phân Đoạn

- ❖ Khi một tiến trình được nạp vào bộ nhớ thì tất cả các đoạn của nó sẽ được nạp vào các phân đoạn còn trống (có thể không liên tiếp nhau) trên bộ nhớ.



ThS. GVC Tô Oai Hùng

85

85

Phân Đoạn

- ❖ Mỗi tiến trình có một bảng phân đoạn (SCT: *Segment Control Table*) riêng.
- ❖ Mỗi phân tử trong bảng phân đoạn gồm tối thiểu 2 trường:
 - base: Địa chỉ khởi đầu của phân đoạn trong bộ nhớ vật lý.
 - limit: Kích thước của phân đoạn, còn được dùng để kiểm soát sự truy xuất bất hợp lệ của các tiến trình.

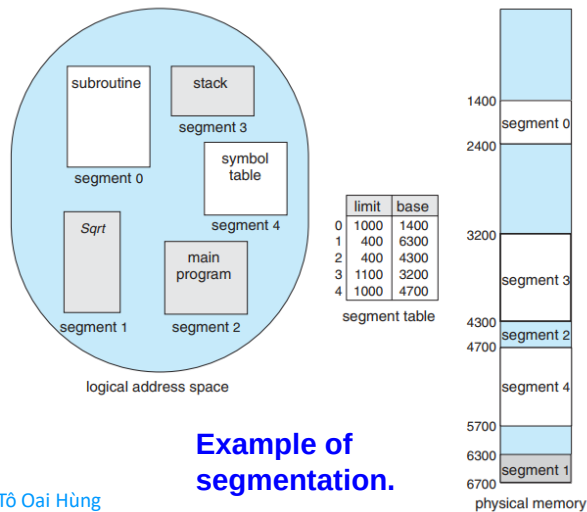
ThS. GVC Tô Oai Hùng

86

86

Phân Đoạn

❖ Ví dụ về phân đoạn:



Example of segmentation.

ThS. GVC Tô Oai Hùng

87

87

Phân Đoạn

- ❖ Các bảng phân đoạn có kích thước nhỏ có thể được chứa trong các thanh ghi.
- ❖ Các bảng phân đoạn có kích thước lớn được chứa trong bộ nhớ chính, HĐH dùng một thanh ghi để lưu trữ địa chỉ bắt đầu nơi lưu trữ bảng phân đoạn (STBR: *Segment Table Base Register*).
- ❖ Thanh ghi STLR (*Segment Table Length Register*) được dùng để ghi kích thước hiện tại của bảng phân đoạn.
- ❖ HĐH dùng một danh sách riêng để theo dõi các segment còn trống trên bộ nhớ.

ThS. GVC Tô Oai Hùng

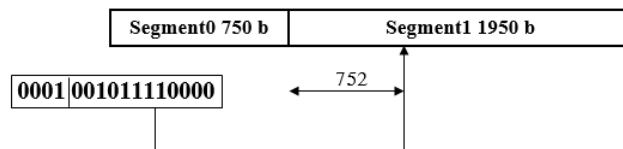
88

88

Phân Đoạn

❖ Địa chỉ phát sinh từ CPU được chia thành hai phần:

- Segment number (s) – số hiệu đoạn: Sử dụng như chỉ mục đến phần tử trong bảng phân đoạn, cho biết số hiệu đoạn tương ứng cần truy xuất.
- Segment offset (o) - địa chỉ tương đối trong đoạn: Giá trị này sẽ được kết hợp với địa chỉ bắt đầu của đoạn để xác định địa chỉ vật lý của ô nhớ cần truy xuất.



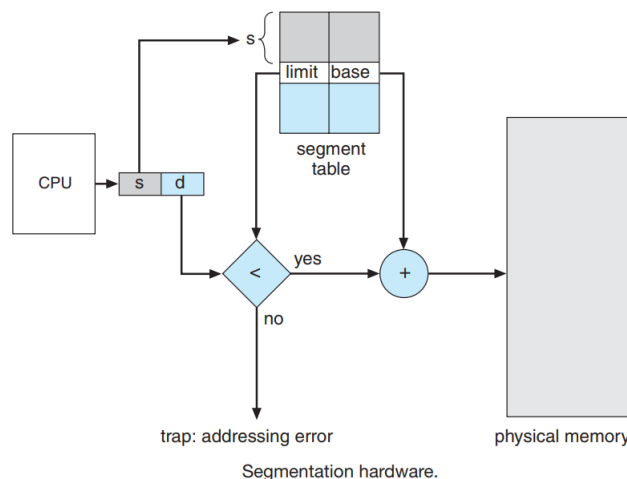
ThS. GVC Tô Oai Hùng

Địa chỉ logic:
Segment # = 1; Offset = 752

89

Phân Đoạn

❖ Phần cứng hỗ trợ chuyển đổi địa chỉ:



ThS. GVC Tô Oai Hùng

90

90

Phân Đoạn

- ❖ Các bước chuyển đổi địa chỉ.
 - n bit trái nhất của địa chỉ logic xác định chỉ số của phân đoạn cần truy xuất trong bảng phân đoạn.
 - Truy xuất phần tử tương ứng thông qua chỉ số trên để tìm địa chỉ vật lý bắt đầu của phân đoạn.
 - So sánh thành phần offset của địa chỉ logic (m bit phải của địa chỉ logic) với trường length của phân đoạn.
 - Nếu offset > length: Địa chỉ truy xuất là không hợp lệ.
 - Ngược lại:

ThS. GVC Tô Oai Hùng

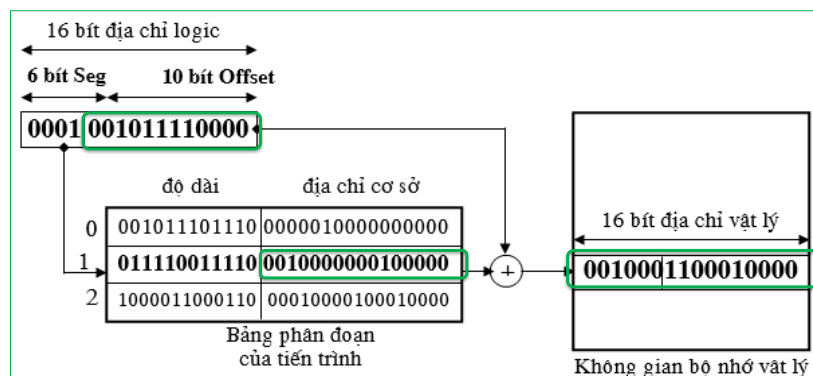
91

91

Phân Đoạn

Địa chỉ vật lý cần tìm = địa chỉ vật lý bắt đầu của phân đoạn + offset.

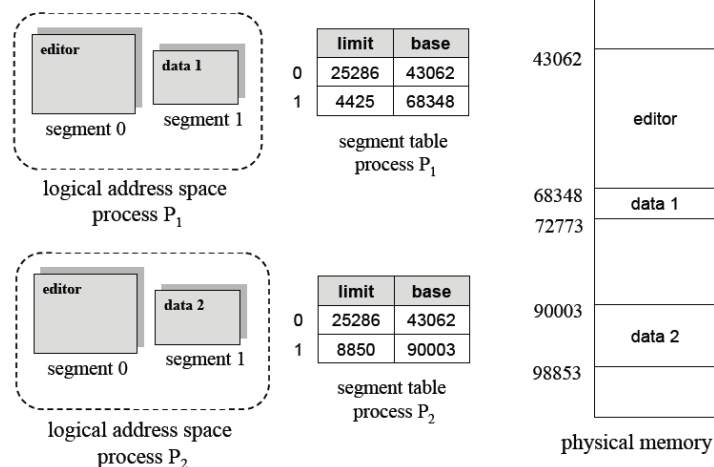
- Giả sử có một địa chỉ logic gồm n + m bit: n = 6 , m = 10 → kích thước tối đa của một segment là 2^{10} byte.



92

Phân Đoạn

❖ Chia sẻ các đoạn:



ThS. GVC Tô Oai Hùng

93

93

Phân Đoạn

❖ Nhận xét:

- Các segment có kích thước không bằng nhau → tương tự phân vùng động.
- Một chương trình có thể chiếm giữ hơn một phân vùng có thể không liền kề nhau.
- Loại trừ được sự phân mảnh nội, vẫn còn phân mảnh ngoại vi (như phân vùng động), tuy nhiên do chương trình được chia nhỏ nên phân mảnh ngoại vi không đáng kể.
- Tường minh đối với người lập trình, cho phép người lập trình tổ chức chương trình và dữ liệu (có thể gán các chương trình và dữ liệu đến các đoạn nhớ khác nhau).

ThS. GVC Tô Oai Hùng

94

94

Phân Đoạn

- Cần giải quyết vấn đề cấp phát động: best-fit, first-fit.
- Segment có kích thước quá lớn có thể không nạp được vào bộ nhớ → Kết hợp phân đoạn với phân trang.

ThS. GVC Tô Oai Hùng

95

95

Phân Đoạn

- ❖ Phân đoạn kết hợp phân trang (Paged segmentation) :
 - Không gian địa chỉ là một tập các phân đoạn, mỗi phân đoạn được chia thành nhiều trang.
 - Khi một tiến trình được đưa vào hệ thống, hệ điều hành sẽ cấp phát cho tiến trình các trang cần thiết để chứa đủ các phân đoạn của tiến trình.
 - Mỗi địa chỉ logic là một bộ ba: $\langle s, p, d \rangle$
 - Số hiệu phân đoạn (s): Sử dụng như chỉ mục đến phần tử tương ứng trong bảng phân đoạn.

ThS. GVC Tô Oai Hùng

96

96

Phân Đoạn

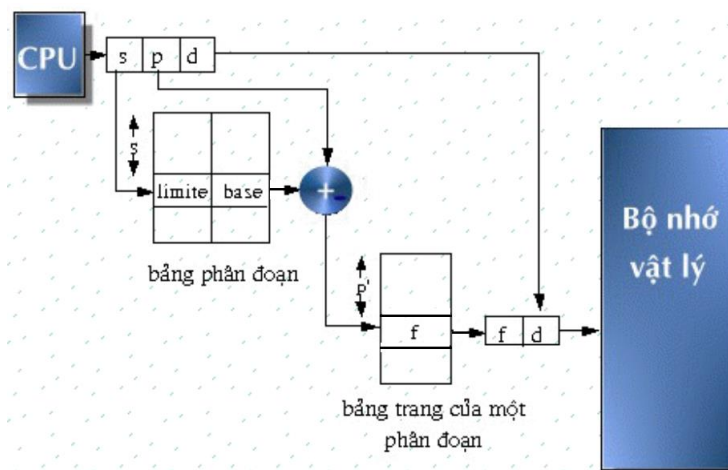
- Số hiệu trang (p): Sử dụng như chỉ mục đến phần tử tương ứng trong bảng trang của phân đoạn.
- Địa chỉ tương đối trong trang (d): Kết hợp với địa chỉ bắt đầu của trang để tạo ra địa chỉ vật lý mà trình quản lý bộ nhớ sử dụng.

ThS. GVC Tô Oai Hùng

97

97

Phân Đoạn



Cơ chế phần cứng của phân đoạn kết hợp phân trang.

ThS. GVC Tô Oai Hùng

98

98

5.5 Mô Hình Quản Lý Bộ Nhớ Ảo

1. Đặc điểm.
2. Kỹ thuật phân trang theo yêu cầu.

ThS. GVC Tô Oai Hùng

99

99

5.5.1 Đặc Điểm

- ❖ Nhìn lại paging và segmentation:
 - Các tham chiếu đến bộ nhớ được chuyển đổi động thành địa chỉ thực lúc process đang thực thi.
 - Một process gồm các phần nhỏ (page hay segment), các phần này được nạp vào các vùng có thể không liên tục trong bộ nhớ chính.
- ❖ Nhận xét:
 - Không phải tất cả các phần của một process cần thiết phải được nạp vào bộ nhớ chính tại cùng một thời điểm:
 - Đoạn mã điều khiển các lỗi hiếm khi xảy ra.

ThS. GVC Tô Oai Hùng

100

100

Đặc Điểm

- Các array, list, table được cấp phát bộ nhớ (cấp phát tĩnh) nhiều hơn yêu cầu thực sự.
- Một số tính năng ít khi được dùng của một chương trình.
- Ngay cả khi toàn bộ chương trình đều cần dùng thì có thể không cần dùng toàn bộ cùng một lúc.

ThS. GVC Tô Oai Hùng

101

101

Đặc Điểm

- ❖ Bộ nhớ ảo (*Virtual memory*).
 - Cơ chế được hiện thực trong hệ điều hành để cho phép thực thi một tiến trình mà chỉ cần giữ trong bộ nhớ chính một phần của không gian địa chỉ luận lý của nó, còn phần còn lại được giữ trên bộ nhớ phụ (đĩa).
 - Cần kết hợp kỹ thuật swapping để chuyển các phần của tiến trình từ bộ nhớ chính ra bộ nhớ phụ và ngược lại khi cần thiết.
- ❖ Ưu điểm của bộ nhớ ảo:
 - Tăng số lượng tiến trình đồng thời được nạp vào bộ nhớ.

ThS. GVC Tô Oai Hùng

102

102

Đặc Điểm

- Một tiến trình có kích thước lớn hơn bộ nhớ thực vẫn có thể thực thi.
- Bộ nhớ ảo cho phép giảm nhẹ công việc của lập trình viên vì không cần bận tâm đến giới hạn của vùng nhớ vật lý.

ThS. GVC Tô Oai Hùng

103

103

Đặc Điểm

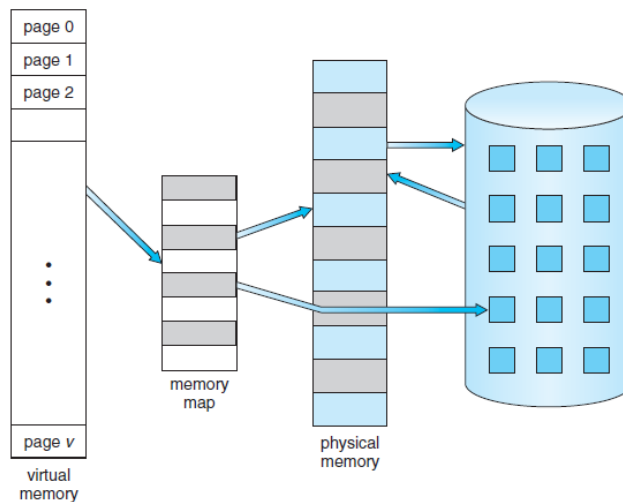


Diagram showing virtual memory that is larger than physical memory.

ThS. GVC Tô Oai Hùng

104

104

5.5.2 Phân Trang Theo Yêu Cầu

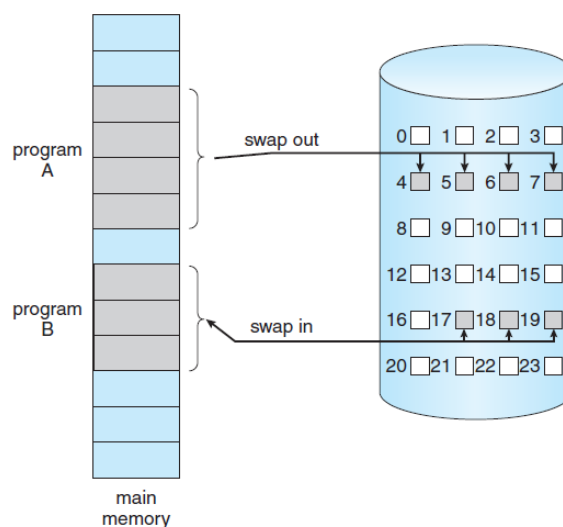
- ❖ Sử dụng kỹ thuật phân trang kết hợp với kỹ thuật swapping.
- ❖ Một tiến trình được xem như một tập các trang, thường trú trên bộ nhớ phụ (đĩa).
- ❖ Khi cần xử lý, tiến trình sẽ được nạp vào bộ nhớ chính, nhưng chỉ nạp những trang cần thiết trong thời điểm hiện tại.
- ❖ → Các trang chỉ được nạp vào bộ nhớ chính khi có yêu cầu, được gọi là phân trang theo yêu cầu (*Demand paging*).

ThS. GVC Tô Oai Hùng

105

105

Phân Trang Theo Yêu Cầu



ThS. GVC Tô Oai Hùng

Transfer of a paged memory to contiguous disk space.

106

106

Phân Trang Theo Yêu Cầu

❖ Cần cung cấp một cơ chế phần cứng giúp phân biệt các trang đang ở bộ nhớ chính và các trang trên đĩa → sử dụng valid bit:

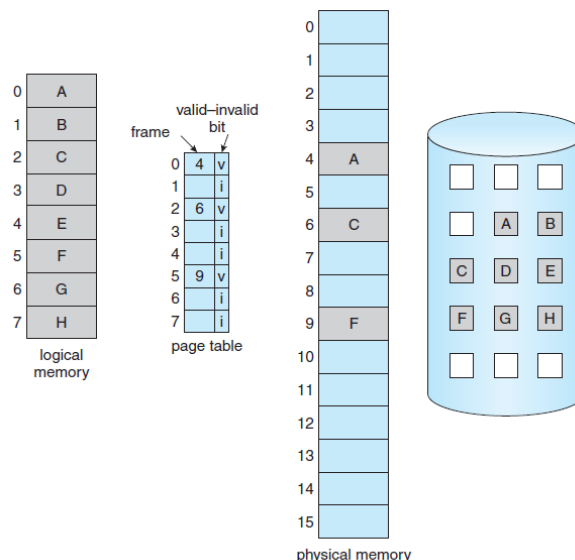
- 1 (valid): trang hợp lệ và đang ở trong bộ nhớ chính.
- 0 (invalid):
 - Trang bất hợp lệ (không thuộc về không gian địa chỉ của tiến trình).
 - Hoặc trang hợp lệ nhưng không nằm trong bộ nhớ chính (đang được lưu trên bộ nhớ phụ).
- Khi trang không nằm trong bộ nhớ chính, phần tử tương ứng trong bảng trang sẽ được đánh dấu invalid và chứa địa chỉ của trang trên bộ nhớ phụ.

ThS. GVC Tô Oai Hùng

107

107

Phân Trang Theo Yêu Cầu



ThS. GVC Tô O

Page table when some pages are not in main memory.

108

108

Phân Trang Theo Yêu Cầu

- ❖ Cơ chế phần cứng: Kết hợp của cơ chế hỗ trợ kỹ thuật phân trang và kỹ thuật swapping.
 - Bảng trang: Phản ánh tình trạng của một trang là đang nằm trong bộ nhớ chính hay bộ nhớ phụ.
 - Bộ nhớ phụ:
 - Thường là đĩa, lưu trữ những trang không được nạp vào bộ nhớ chính.
 - Vùng không gian đĩa dùng để lưu trữ tạm các trang trong kỹ thuật swapping được gọi là không gian swapping.

ThS. GVC Tô Oai Hùng

109

109

Phân Trang Theo Yêu Cầu

- ❖ Lỗi trang: Khi có một tham chiếu đến một trang không có trong bộ nhớ chính (valid bit = 0) → phần cứng phát sinh một ngắt (*page-fault trap*) báo cho HĐH xử lý:
 - Kiểm tra truy xuất đến bộ nhớ là hợp lệ hay không hợp lệ.
 - Nếu truy xuất không hợp lệ, kết thúc tiến trình.
 - Ngược lại:
 - Tìm vị trí chứa trang muốn truy xuất trên đĩa.
 - Tìm một khung trang trống trong bộ nhớ chính → chuyển trang muốn truy

ThS. GVC Tô Oai Hùng

110

110

Phân Trang Theo Yêu Cầu

xuất từ bộ nhớ phụ vào bộ nhớ chính bằng cách nạp trang cần truy xuất vào khung trang trống đã chọn, cập nhật nội dung bảng trang, bảng khung trang tương ứng

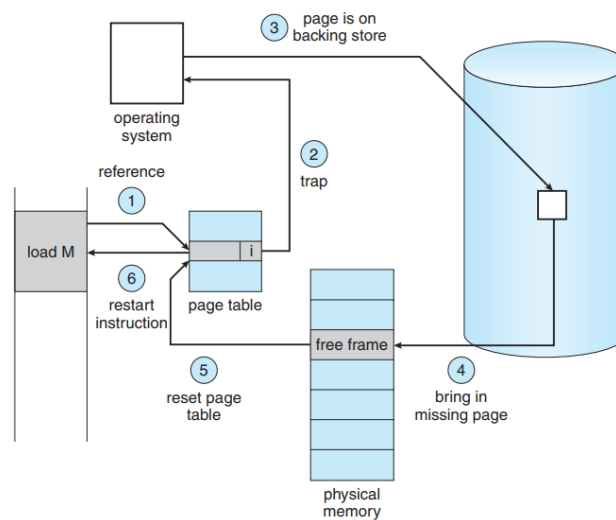
- Nếu không còn khung trang trống → thay thế trang.

ThS. GVC Tô Oai Hùng

111

111

Phân Trang Theo Yêu Cầu



ThS. GVC Tô Oai Hùng

Steps in handling a page fault.

112

112

Phân Trang Theo Yêu Cầu

❖ Copy-on-Write:

- Là một kỹ thuật phổ biến được sử dụng trong nhiều hệ điều hành (Windows XP, Linux, và Solaris).
- Cho phép tiến trình cha và con cùng chia sẻ các trang trong bộ nhớ khi mới khởi tạo tiến trình con.
- Các trang được chia sẻ này được đánh dấu là các trang copy-on-write, nếu một trong hai tiến trình muốn ghi vào một trang được chia sẻ, một bản sao của trang đó sẽ được tạo.
- Chỉ những trang có thể sửa đổi mới cần được đánh dấu là copy-on-write.

ThS. GVC Tô Oai Hùng

113

113

Phân Trang Theo Yêu Cầu

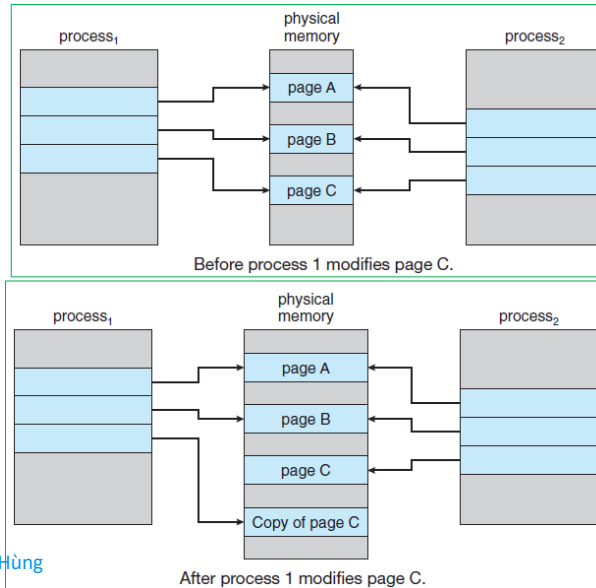
- Tạo tiến trình hiệu quả hơn: Chỉ các trang bị sửa đổi mới được copy.
- Khi tạo bản sao cho một trang trong copy-on-write, HĐH cần cấp phát các trang rồi để lưu các trang copy này.
- Các trang rồi được cấp phát từ một tập hợp các trang được xóa trắng trước đó.
- Một ứng dụng của copy-on-write là được sử dụng ở lời gọi hàm fork().

ThS. GVC Tô Oai Hùng

114

114

Phân Trang Theo Yêu Cầu



ThS. GVC Tô Oai Hùng

115

115

Phân Trang Theo Yêu Cầu

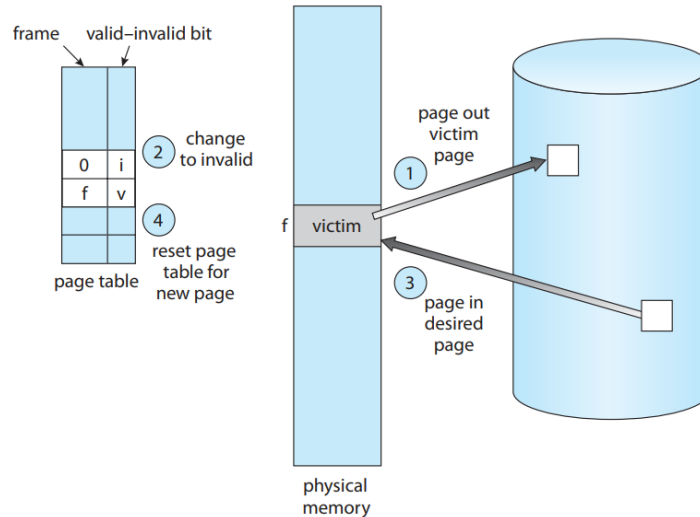
- ❖ Thay thế trang: Khi đưa một trang từ bộ nhớ ngoài vào bộ nhớ chính, trường hợp không tìm được frame trống:
 - Chuyển một trang trong bộ nhớ chính ra bộ nhớ ngoài và nạp một trang khác vào bộ nhớ chính.
 - Dùng một giải thuật thay trang để chọn một trang hy sinh (*victim page*).
 - Ghi *victim page* lên bộ nhớ ngoài.
 - Đọc trang đang cần vào frame trống (thay vào chỗ *victim page*).

ThS. GVC Tô Oai Hùng

116

116

Phân Trang Theo Yêu Cầu



ThS. GVC Tô Oai Hùng

117

117

Phân Trang Theo Yêu Cầu

❖ Thay thế trang (tt):

- Trong mỗi phần tử của bảng phân trang, sử dụng một bit cập nhật (dirty bit) để phản ánh tình trạng trang có bị sửa đổi hay không.
 - dirty bit = 1: Nội dung trang đã bị sửa đổi
 - Khi cần thay thế một trang, nếu dirty bit = 1 → trang cần được lưu lại trên đĩa, ngược lại (trang không bị thay đổi) → không cần lưu trữ trang trở lại đĩa.

ThS. GVC Tô Oai Hùng

118

118

Phân Trang Theo Yêu Cầu

- Các thuật toán thay thế trang cần phải đạt mục tiêu là chọn victim page sao cho sau khi thay thế sẽ gây ra ít lỗi trang nhất.
- Thuật toán thay thế trang được đánh giá hiệu quả bằng cách xử lý trên một chuỗi các địa chỉ cần truy xuất và tính toán số lượng lỗi trang phát sinh.

ThS. GVC Tô Oai Hùng

119

119

Phân Trang Theo Yêu Cầu

❖ Giải thuật thay trang FIFO:

- Chọn trang ở trong bộ nhớ lâu nhất.
- Ví dụ: Sử dụng 3 khung trang, ban đầu cả 3 đều trống.

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2																
	0	0	0		2	2	4	4	4	0			0	0			7	7	7
					3	3	3	2	2	2			1	1			1	0	0
		1	1		1	0	0	0	3	3			3	2			2	2	1

page frames

FIFO page-replacement algorithm.

ThS. GVC Tô Oai Hùng

120

120

Phân Trang Theo Yêu Cầu

❖ Giải thuật thay trang FIFO (tt):

- Dễ hiểu, dễ cài đặt.
- Có thể cho kết quả không tốt: trang được chọn để thay thế có thể là trang chứa nhiều dữ liệu cần thiết, thường xuyên được sử dụng nên được nạp sớm, do vậy khi bị chuyển ra bộ nhớ phụ sẽ nhanh chóng gây ra lỗi trang.
 - Số lượng lỗi trang xảy ra sẽ tăng lên khi số lượng khung trang nhiều → nghịch lý Belady.
 - Ví dụ chuỗi tham chiếu 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.
 - 3 khung trang: 9 lỗi.
 - 4 khung trang: 10 lỗi.

ThS. GVC Tô Oai Hùng

121

121

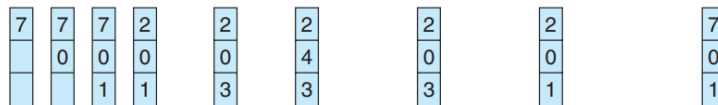
Phân Trang Theo Yêu Cầu

❖ Giải thuật thay trang OPT (Optimal):

- Thay thế trang sẽ được tham chiếu trễ nhất trong tương lai.
- Ví dụ: Sử dụng 3 khung trang, khởi đầu đều trống.

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



page frames

Optimal page-replacement algorithm.

- Số lượng lỗi trang thấp nhất, không xảy ra nghịch lý Belady.
- Không khả thi, vì không thể biết trước chuỗi truy xuất của tiến trình!

ThS. GVC Tô Oai Hùng

122

122

Phân Trang Theo Yêu Cầu

❖ Giải thuật thay trang Least Recently Used (LRU):

- Thay thế trang nhớ không được tham chiếu lâu nhất.
- Ví dụ: Sử dụng 3 khung trang, khởi đầu đều trống.

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0		1		1		1			
	0	0	0		0		0	0	3	3		3		0		0			
		1	1		3		3	2	2	2		2		2		7			

page frames

LRU page-replacement algorithm.

ThS. GVC Tô Oai Hùng

123

123

Phân Trang Theo Yêu Cầu

❖ LRU (tt):

- Cần được cơ chế phần cứng hỗ trợ để xác định thứ tự cho các trang theo thời điểm truy xuất cuối cùng. Có hai cách:

1. Sử dụng bộ đếm:

- Mỗi phần tử trong bảng trang được bổ sung một trường ghi nhận thời điểm truy xuất mới nhất.
- Bổ sung vào cấu trúc của CPU một bộ đếm mỗi lần có sự truy xuất bộ nhớ, giá trị của counter tăng lên 1.
- Mỗi lần thực hiện truy xuất đến một trang, ghi giá trị của counter vào một

ThS. GVC Tô Oai Hùng

124

124

Phân Trang Theo Yêu Cầu

trường lưu thời điểm truy xuất mới nhất của phần tử tương ứng với trang trong bảng trang.

- Thay thế trang có giá trị trường lưu thời điểm truy xuất mới nhất là nhỏ nhất.

2. Sử dụng stack:

- Tổ chức một stack lưu trữ các số hiệu trang.
- Mỗi khi thực hiện một truy xuất đến một trang, số hiệu của trang sẽ được xóa khỏi vị trí hiện hành trong stack và đưa lên đầu stack.
- Trang ở đỉnh stack là trang được truy xuất gần nhất, và trang ở đáy stack là trang lâu nhất chưa được sử dụng.

ThS. GVC Tô Oai Hùng

125

125

Phân Trang Theo Yêu Cầu

reference string

4 7 0 7 1 0 1 2 1 2 7 1 2

2
1
0
7
4

stack
before
a

7
2
1
0
4

stack
after
b

↑ ↑
a b

Use of a stack to record the most recent page references.

ThS. GVC Tô Oai Hùng

126

126